Toward Efficient Distributed Network Security: A Lightweight Multitask Traffic Analysis Framework

Jiadong Fu, Jiang Fang, Jiyan Sun, Shangyuan Zhuang, Yinlong Liu, and Zhiqiang Lv

***Abstract*—With the rapid development of cloud computing, network architectures are moving towards distributed computing, which performs data processing at edge nodes to reduce latency, enabling more efficient and scalable network services. Nevertheless, this shift introduces significant security challenges due to the heterogeneity of communications protocols and the vulnerabilities of edge devices. To effectively secure these distributed networks, it is essential to perform multiple traffic analysis tasks, e.g. Network Intrusion Detection, Encrypted Traffic Classification, and Application Traffic Classification. However, existing methods have limited generic feature extraction and require the deployment of multiple models to solve multiple tasks, which exceeds the resource capacity of edge nodes. To address these challenges, we introduce a Lightweight Multitask Traffic Analysis Framework LiMTa, which novelly proposes a traffic pre-training method, FreqRec, and a lightweight multi-task model fine-tune method, MT-Adapter. FreqRec enables high-level semantic feature extraction by reconstructing the frequency features of traffic samples, and MT-Adapter efficiently performs multiple tasks by computing the pre-trained model only once. Experimental results demonstrate that our approach achieves state-of-the-art (SOTA) performance on six traffic analysis tasks. Moreover, the MT-Adapter module only fine-tunes a small number of parameters, accounting for only 6.37% of the pre-trained model's parameters, and achieves the same result as the full fine-tuning. Compared to full fine-tuning, LiMTa reduces the time cost by 50.9% and the space cost by 57.4% in six edge traffic analysis tasks.**

***Index Terms*—Edge security, frequency feature, feature reconstruction, reuse model, multitask traffic analysis task.**

I. Introduction

With the rapid development of the Internet of Things (IoT) and mobile communication technologies, network architectures are shifting from traditional centralized processing to distributed network computing [1], [2], [3].

Received 22 January 2025; revised 8 December 2025; accepted 8 December 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor A. Khreishah. Date of publication 17 December 2025; date of current version 12 January 2026. This work was supported in part by the Climbing Program of the Institute of Information Engineering, Chinese Academy of Sciences, under Grant E3Z0031. *(Corresponding authors: Jiang Fang; Yinlong Liu.)*

Jiadong Fu, Shangyuan Zhuang, Yinlong Liu, and Zhiqiang Lv are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: fujiadong@iie.ac.cn; zhuangshangyuan@iie.ac.cn; liuyinlong@iie.ac.cn; lvzhiqiang@iie.ac.cn).

Jiang Fang is with the Electronic Engineering Institute, National University of Defense Technology, Hefei 230037, China, and also with the Jianghuai Advance Technology Center, Hefei 230031, China (e-mail: jiangfang2025@gmail.com).

Jiyan Sun is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China (e-mail: sunjiyan@iie.ac.cn).

Digital Object Identifier 10.1109/TON.2025.3643832

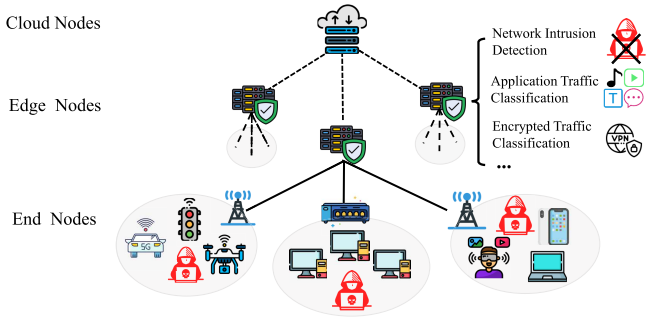


Fig. 1. Edge traffic analysis tasks in distributed network architectures.

Benefiting from data processing at edge nodes close to the data source, distributed networks can significantly reduce latency for real-time applications and alleviate bandwidth constraints typically existing in centralized networks. As shown in Figure 1, edge nodes are tasked with managing many kinds of edge devices and network services in their respective areas. Therefore, edge nodes are required to handle network traffic from heterogeneous protocols of edge devices [4], such as MQTT and CoAP. In addition, many edge devices have potential security vulnerabilities due to insufficient security design and testing [5], [6]. These security vulnerabilities expose distributed networks to significant security risks [7], [8], e.g., Botnet attacks targeting IoT devices Mirai [9], IoT Goes Nuclear [10], and large-scale DDoS attacks [11].

These security risks can lead to significant downtime, loss of trust from users, and substantial financial and reputational damage for service providers.

To ensure the security of distributed networks, it is essential to perform multiple traffic analysis tasks, such as Network Intrusion Detection [12], Encrypted Traffic Classification [13], and Application Traffic Classification [14], for effective detection of network security risks. By labeling data and training specific models for specific traffic analysis tasks, current machine learning [15], [16] and deep learning [17], [18] methods have achieved good performance. Furthermore, to ensure timely task completion and minimize the risk of privacy breaches, these traffic analysis tasks are advised to be trained and deployed at edge nodes closer to the traffic source [19]. However, the above machine learning and deep learning methods are primarily designed for single-task scenarios, making it necessary to train and deploy multiple models to handle multiple traffic analysis tasks [20]. Since edge nodes are often limited in computational and storage capacities, the resource demands of multiple models greatly exceed the capacity of

edge nodes. This limitation makes these methods impractical when applied to edge nodes to solve multiple traffic analysis tasks [21], [22]. Besides, these methods heavily rely on large amounts of labeled data and are restricted to single-task scenarios, rendering them both time-intensive and impractical for multi-task scenarios [23], [24].

Recently, the "*Pre-training*" & "*Fine-tuning*" paradigm has been applied to address the common challenges of label scarcity and poor task transferability [23], [24], [25]. This paradigm has shown great potential natively in multiple traffic analysis tasks for distributed networks [26]. In this paradigm, a massive amount of unlabeled network traffic is first collected from the open Internet to ensure that the model pre-training process does not involve any private information. After that, the pre-training process with huge computational and storage requirements is performed on massive unlabeled datasets in the cloud to train a feature extraction model. The pre-trained model is then distributed to edge nodes, where only fine-tuning a small amount of labeled data is required for the model to perform traffic analysis tasks.

However, these existing traffic analysis methods for "*Pre-training*" & "*Fine-tuning*" face two serious challenges when applied to edge nodes. Firstly, in the pre-training stage, existing pre-training methods retain excessively low-level semantics and overlook critical frequency features, limiting the model's performance in downstream tasks. In detail, these methods are usually trained on raw sample reconstruction and prediction tasks based on *temporal features* of traffic [27], e.g., autoencoder reconstruction [25] and context prediction [23] and have achieved remarkable performance. While these methods focus on temporal features, ignoring the potential period and *frequency features* in the traffic data [28], which is critical for detecting periodic attacks. Furthermore, these methods reconstruct temporal traffic features in raw space, retaining a lot of unnecessary low-level semantic traffic information [29], e.g. non-meaningful padding bytes, and are inefficient in training, affecting the performance of the model in downstream traffic analysis tasks [28], [30], [31].

Secondly, in the fine-tuning stage, existing methods for analyzing network traffic, mainly including linear fine-tuning, full fine-tuning, and parameter-efficient fine-tuning [32], present issues of limited adaptability or computational redundancy, limiting the application of the model on edge nodes. For linear fine-tuning, this method can update only the parameters of the classification layer to adapt to different traffic analysis tasks. Nevertheless, the limited adaptability of the model makes it difficult to achieve satisfactory performance in different traffic analysis tasks. For full fine-tuning, this method can update all pre-trained model parameters to enable exceptional performance on specific tasks. While updating all parameters of the pre-trained model requires a huge resource cost, it is very difficult to train and apply a pre-trained model on edge nodes. For parameter-efficient fine-tuning like LoRA [33], it can achieve excellent model performance by adding LoRA parameters to the hidden layer of the pre-trained model. Yet, the output of the pre-trained model's hidden layer relies on the computation of the LoRA layer. This leads to that, in the inference stage, the parameter-efficient fine-tuning algorithm, similar to full fine-tuning, requires multiple forward computations on the pre-trained model to combine the different LoRA layer outputs when dealing with multi-traffic analysis tasks. Multiple forward computations on resource-constrained edge nodes significantly increase the processing time and can't timely handle multiple traffic analysis tasks.

To address the above challenges, we propose a **Li**ghtweight **M**ultitask **T**raffic **A**nalysis Framework, **LiMTa**. First, in the pre-training stage, we propose a novel traffic pre-training method, FreqRec, which trains the model on the feature space through the frequency feature reconstruction task. Specifically, FreqRec constructs frequency feature-missing samples by randomly masking frequency features. Then, it extracts high-level semantic features by designing a dual-stream network to reconstruct the frequency features of feature-missing samples in the feature space. This method enables the model to effectively extract temporal and frequency features of traffic without relying on any labeled samples. Experimental results show that FreqRec achieves SOTA performance in various traffic analysis tasks.

Second, in the fine-tuning stage, we propose a lightweight multi-task model fine-tuning method, MT-Adapter. Unlike parameter-efficient fine-tuning and full fine-tuning methods, the MT-Adapter is independent of the pre-trained model, without affecting the pre-trained model's forward process and pre-trained parameters. Our method requires only one computation of the pre-trained model to obtain hidden layer features and then feeds these features into different lightweight MT-Adapters to achieve the same model performance as full fine-tuning in solving multiple tasks. Moreover, the MT-Adapter can efficiently perform different tasks by adding and fine-tuning only a small portion of parameters, just 6.37% of the pre-trained model's parameters. Our method significantly reduces the resource overhead of model deployment and inference. Our method surpasses the pre-training methods ET-BERT [23] and YaTC [24] by 5.03% and 7.47% on the CrossPlatform-Android dataset, respectively. Moreover, compared to full fine-tuning, LiMTa reduces the time cost by 50.9% and the space cost by 57.4% in six edge traffic analysis tasks. In conclusion, the main contributions of this paper are as follows:

1) We propose a lightweight pre-training and fine-tuning framework, LiMTa, which is designed to perform multiple traffic analysis tasks on edge nodes with limited computational resources.
2) We propose a novel traffic pre-training method, FreqRec, which achieves a high-level semantic feature extraction model by reconstructing the frequency features of traffic samples. This way effectively extracts the frequency and temporal features of traffic data.
3) We introduce a pre-trained model fine-tuning method, MT-Adapter, which efficiently performs multiple tasks by computing the pre-trained model only once. This pre-trained model reuse way effectively reduces the training and inference computational overhead.
4) We conducted extensive experiments on six traffic-related security tasks, providing overall comparisons and detailed evaluations. The experimental results

demonstrate that our method achieves state-of-the-art (SOTA) performance.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the detailed design of LiMTa. Section IV provides an experimental evaluation of its performance. Section V discusses the limitations of LiMTa. Finally, Section VI concludes the paper.

II. Related Work

In this section, we review the main research related to our work. It mainly includes machine learning-based traffic analysis methods, deep learning-based traffic analysis methods, and recent emerging pre-training and fine-tuning traffic analysis methods.

A. Machine Learning Based Traffic Analysis Methods

Recently, machine learning-based traffic analysis methods have achieved excellent traffic analysis performance by constructing classification or regression models to extract features from traffic data [34]. These methods include classic algorithms such as Support Vector Machine (SVM), Random Forest, and K-Nearest Neighbour (KNN), etc [35]. Typically, AppScanner [16] extracts traffic features and combines them with a variety of machine learning models to achieve efficient classification of mobile application traffic and detection of malicious attacks, and exhibits low false alarm rates even in encrypted communication environments. FlowPrint [15] analyzes the metadata of a flow and extracts the feature fingerprint of each flow to accurately identify the traffic patterns of an application or service with the help of machine learning models. However, machine learning methods rely on high-quality manual feature engineering; their performance degrades when faced with complex traffic patterns and sparsely characterized data [36]. In addition, since their models are usually designed for specific tasks, they are difficult to support multi-task traffic analysis requirements.

B. Deep Learning Based Traffic Analysis Methods

Deep learning-based traffic methods are able to automatically extract high-dimensional feature representations from raw traffic data through multi-layer neural networks without relying on manual feature engineering. This makes deep learning methods highly generalizable when dealing with complex traffic patterns, encrypted traffic, and unknown attacks [37]. Typical deep learning architectures include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Self-Attention Networks (Transformers), etc. DeepPacket [18] uses CNNs to capture local features of network traffic and combines them with RNNs to capture temporal information in traffic sequences to achieve efficient and accurate traffic classification. This joint neural network modeling approach significantly improves the ability to recognize complex traffic patterns. FS-Net [17] employs CNN model to extract deep traffic features and filters the most relevant features for the classification task with an adaptive feature selection module. This approach automatically identifies the most discriminative features, thereby improving the accuracy and efficiency of traffic analysis. However, deep learning-based methods usually require large amounts of labeled data and a high level of computational resources for the training process [36]. In addition, deep learning-based approaches accomplish multiple traffic analysis tasks that require training, deploying, and reasoning about multiple deep model parameters, further expanding the demand for computational resources [26]. These methods are often unavailable on edge nodes with limited computational resources.

C. Pre-Training & Fine-Tuning Based Traffic Analysis Methods

The pre-training methods typically construct self-supervised learning tasks to learn generalizable feature representations from large-scale unlabeled datasets. Then, a small amount of labeled traffic data is used to fine-tune it for a specific downstream task. This approach significantly reduces the need for labeled data. Moreover, the "*Pre-training*" & "*Fine-tuning*" paradigm can be effectively applied to distributed network architectures [26]. The resource-intensive pre-training process is performed on massive unlabeled datasets in the cloud. The pre-trained model is then distributed to the edge nodes. With a small amount of labeled data for fine-tuning, the model is able to perform a series of traffic analysis tasks at the edge.

In recent years, some research has explored pre-training methods for traffic analysis tasks. Specifically, the PERT framework [38] utilized dynamic word embedding techniques to automatically extract traffic features from the contextual distribution of traffic payload bytes. Subsequently, it leverages the pre-trained network for classification tasks to achieve greater performance. Consequently, the ET-BERT model [23] adapts the BERT architecture from NLP to learn deep traffic representations, showing improved performance in traffic classification tasks. Similarly, YaTC [24] adopt autoencoder-based architectures, where the pre-training objective is to reconstruct raw traffic features. With the proposal of mamba technology, NetMamba [25] applies a mamba-based architecture for traffic feature extraction, by reconstructing raw traffic features, demonstrating the potential of self-supervised learning on unlabeled data. Over the past year, TrafficFormer [39] utilizes a Transformer encoder architecture and is pre-trained using a fine-grained multi-classification task, demonstrating significant performance gains and competitive computational efficiency across various traffic classification benchmarks. MIETT [40] leverages the Transformer to model dependencies across multiple related traffic instances within a flow. This work highlights the trend of using complex Transformer variants to capture intricate behavioral dependencies.

However, these methods reconstruct raw traffic information and retain massive unnecessary low-level semantic information, affecting the performance of the model in performing downstream traffic analysis tasks. Moreover, these methods focus on temporal features, ignoring the potential period and frequency features in the traffic data, which are critical for detecting periodic attacks. Besides, full fine-tuning and linear fine-tuning strategies face challenges in terms of high computational costs and poor adaptability, respectively. Additionally,

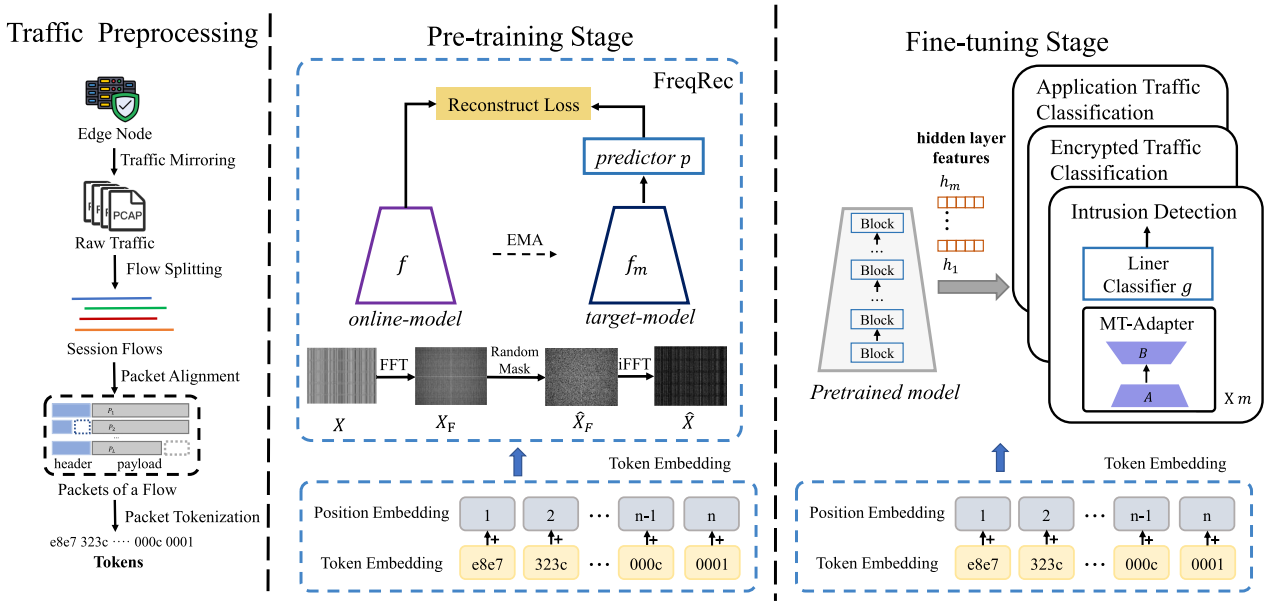


Fig. 2. LiMTa framework. The framework consists of three stages: (Left) traffic preprocessing, (Center) pre-training with FreqRec, and (Right) task-specific fine-tuning with MT-Adapter.

in the inference stage, the parameter-efficient fine-tuning algorithm needs to perform multiple forward computations on the pre-trained model. This approach also fails to address the reusability of the pre-trained models in multi-tasking scenarios. Thus, it is difficult to apply existing methods to edge nodes to solve multiple traffic analysis tasks.

III. Framework Design

This section describes the proposed **Li**ghtweight **M**ultitask **T**raffic **A**nalysis Framework, **LiMTa**. We first describe the overall architecture of LiMTa and then detail its various stages.

A. Framework Overview

In this subsection, we detail our proposed framework LiMTa. As shown in Figure 2, the LiMTa consists of three main stages: traffic preprocessing stage, pre-training stage, and fine-tuning stage. Specifically, in the traffic preprocessing stage, LiMTa first divides the raw traffic data into multiple independent session flows based on five-tuple features. Secondly, to ensure the consistency of session flows, LiMTa aligns the header and payload length of each session flow packet. Finally, the aligned packets are concatenated into a continuous binary-byte sequence and divided into tokens of a fixed size. This approach transforms raw traffic into structured, tokenized data, simplifying the data while preserving essential relationships.

In the pre-training stage, we propose a novel traffic pre-training method, FreqRec, which pre-trains powerful traffic feature extraction models by doing a frequency feature reconstruction task. Specifically, LiMTA first embeds tokenized traffic data into a vector representation as the original traffic sample. Next, FreqRec computes the frequency feature samples by Fast Fourier Transform and then constructs the frequency missing samples by randomly masking the frequency features. Finally, FreqRec constructs a dual-stream network to compute the feature representations of the original traffic samples and the frequency-missing samples. Then, FreqRec implements the reconstruction of the frequency features in the high-level feature space via *predictor*. Experiments verify that this design can pre-train powerful feature extraction models from massive unlabeled traffic data.

In the fine-tuning stage, we novelly introduce a pre-trained model fine-tuning method called MT-Adapter. The MT-Adapters are separated from the pre-trained models by computing the pre-training model to obtain the hidden layer features only once, and then feeding these features into different lightweight MT-Adapters, thus executing multiple tasks efficiently. Specifically, LiMTA freezes the parameters of the pre-trained model and adds lightweight MT-Adapter modules for each traffic analysis task. Then, LiMTA embeds the raw token data into vector representations and feeds them into the pre-trained model to compute hidden layer features. These features are then passed to different MT-Adapter modules to generate predictions for various tasks. This design enables effective reuse of the pre-trained model, significantly reducing computational overhead in multitasking scenarios and making it highly suitable for deployment on edge nodes.

B. Traffic Preprocessing Stage

This subsection details the traffic preprocessing stage in LiMTa. Inspired by [23] and [25], the traffic preprocessing stage in this paper includes traffic mirroring, data flow segmentation, packet alignment, and packet tokenization.

1) Traffic Mirroring and Flow Splitting: Traffic mirroring operates by duplicating packets during their transmission through the switch and forwarding these copies to a designated mirror port. In this study, a port forwarding strategy is configured on the core switch to route all traffic packets passing through edge nodes to the port on which the traffic analysis model is processed. The traffic at the edge node comprises

a mix of packets from various devices and hosts operating across different protocols. To minimize interference from non-host traffic, LiMTa first filters out packets associated with IP-independent protocols. Then, packets with the same five-tuple (source IP, destination IP, source port, destination port, protocol) are aggregated into a flow. Each flow represents a complete network communication process between hosts. The flow encapsulates critical information about host interactions and key communication features, e.g. traffic load, protocol usage, and the behavioral patterns of both communicating parties. Overall, this data provides a comprehensive view of the network's operational status and communication patterns.

2) Packet Alignment: First, to eliminate potential correlations between packets, LiMTa anonymizes all packets by masking Ethernet headers and IP addresses. Then, LiMTa considers the variation in byte length between different protocol packets, such as MQTT [41] packets have a 2-byte header and typically a 2-byte to several hundred-byte payloads, while UDP packets have a fixed 8-byte header and a maximum payload length of 65,507 bytes. Packets that are either too long or too short may affect the byte information of the flow. Therefore, it is essential to standardize packet sizes by allocating a uniform size to all packets and assigning fixed lengths to the headers and payloads of the packets. Thus, LiMTa trims packets that are too long, while shorter packets are padded with "0" bytes to ensure that both the header and payload of the packets have the same length, denoted as N_h bytes and N_p bytes, respectively. In this paper, we set N_h to 80 and N_p to 240 bytes. The choice of $N_h = 80$ and $N_p = 240$ for packet alignment is based on the empirical finding that most critical protocol headers are encapsulated within the first 80 bytes, and the initial 240 bytes of the payload are highly discriminative, containing key application-layer fingerprints (e.g., TLS/HTTP handshake messages) crucial for classification.

3) Packet Tokenization: To collect more comprehensive traffic information, we follow the method outlined in previous research [25], dividing multiple bytes into a single token. Specifically, LiMTa first aggregates the bytes of k consecutive packets from each flow into a unified byte array. It is defined as $F_b = [b_1, b_2, \ldots, b_{L_b}]$, where $L_b = k \times (N_h + N_p)$ represents the length of the array, and b_i represents the i-th byte. Secondly, to improve computational efficiency, we group N_t adjacent binary characters into a single token t, so each byte array is divided into $N = L_b/N_t$ tokens. In this paper, we set N_t to 16. The tokenisation size of $N_t = 16$ is selected as an optimal trade-off to capture the necessary local semantic dependencies while significantly reducing the sequence length (L) fed into the Transformer, thereby ensuring computational efficiency for latency-critical edge deployments. Finally, the tokens contained in a flow can be represented as $F_t = [t_1, t_2, \ldots, t_N]$. This method can reduce the model input's length while retaining essential sequential information in the traffic data.

C. Pre-Training Stage

In this stage, to address the issue that existing methods retain excessively low-level semantics and overlook critical frequency features, We propose a novel pre-training method called FreqRec, which trains powerful traffic feature extraction models by doing a frequency feature reconstruction task.

Specifically, FreqRec first embeds the token array F_t to acquire the original sample vector X. Then, based on the original sample vector X, FreqRec constructs the frequency feature-missing samples vector $\hat{X}$. After that, FreqRec builds a dual-stream network to compute the feature representations of the original traffic samples and the frequency feature-missing samples. Finally, FreqRec reconstructs the frequency feature-missing samples through the projection layer and then aligns the original sample feature representations with the reconstructed frequency feature-missing sample feature representations in the high-level feature space. This process completes a frequency feature reconstruction task. Through the FreqRec pre-training method, LiMTa can pre-train a generic traffic feature extraction model for multiple traffic analysis tasks.

1) Token Embedding: Since the original token data is typically high-dimensional and sparse [24], [25], we use token embedding and position embedding to represent the characteristics of each token. This allows the token to be compressed into a representation while retaining the semantic relationships of the traffic data.

Specifically, given a traffic token sequence F_t, we first apply token embedding to obtain a vector representation of the tokens, which projects each token into a vector of size d. For each token $t_i \in F_t$, it is mapped into a vector representation using an embedding matrix E_t with dimensions $N_t \times d$, denoted as $e_i = E_t(t_i) = t_i * E_t$. Consequently, we add a position embedding to each token to capture the positional information of tokens within the sequence. For the traffic token sequence F_t of length N, we use a position embedding matrix E_p with dimensions $N \times d$ to represent the embedding of different positions. The position embedding can be expressed as $p_i = E_p(i)$. The final embedding result x_i is the sum of the token embedding and the position embedding, denoted as Equation 1.

$$x_i = e_i + p_i = E_t(t_i) + E_p(i) \tag{1}$$

In this work, we set the embedding dimension $d = 512$. For a given traffic token sequence $F_t = [t_1, t_2, \ldots, t_N]$, after the embedding process, we obtain $X = \{x_1, x_2, \ldots, x_N\}$. X represents the traffic temporal feature vector.

2) Frequency Feature-Missing Samples Construction: To comprehensively extract the high-level semantic features of traffic data, FreqRec first constructs the frequency feature missing samples by randomly masking the frequency features. For the two-dimensional traffic feature X obtained after token embedding, we first compute the frequency features X_F of X using the Fast Fourier Transform (FFT) [42], denoted as Equation 2, to obtain the frequency feature information of the traffic samples.

$$X_F[k_1, k_2] = \sum_{i=0}^{N-1} \sum_{j=0}^{d-1} X[i, j] e^{-j2\pi\left(\frac{ik_1}{N} + \frac{jk_2}{d}\right)} \tag{2}$$

where $X[i, j]$ denotes the element in the i-th row and j-th column of the feature X, and $X_F[k_1, k_2]$ represents the

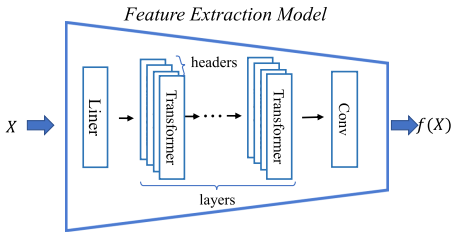


Fig. 3. Architecture of the traffic feature extraction model used as the pre-trained backbone in LiMTa.

frequency component in the frequency-domain matrix at the k_1-th row and k_2-th column. The terms $e^{-j2\pi\frac{ik_1}{N}}$ and $e^{-j2\pi\frac{jk_2}{d}}$ correspond to the Fourier transform factors for the rows and columns, respectively.

Subsequently, a Random Mask process is applied to X_F. The Random Mask process generates a random 0-1 matrix M and dot-products it with the original frequency feature matrix to compute the frequency feature-missing sample. To ensure the effectiveness of model training, we set the mask ratio to 25%, as section IV-G shows it offers the best trade-off between reconstruction difficulty and feature preservation. This involves setting certain frequency components to zero, thereby generating samples that exhibit feature differences compared to the original samples. The Random Mask process is defined as Equation 3.

$$\hat{X}_F = X_F \odot M; M \in \{0,1\}^{N\times d} \tag{3}$$

where $\odot$ represents the dot product. Compared with raw temporal features, frequency-domain representations emphasize periodicity and global temporal regularities in the traffic. Typical examples include periodic keep-alives, IoT telemetry uploads, botnet beaconing, and on–off patterns in streaming or DoS attacks, which often become more salient at specific frequencies after the transformation.

Since the model inputs are temporal features, It needs to transform it back to the temporal domain using the inverse Fast Fourier Transform (iFFT) [42], resulting in the frequency feature-missing sample $\hat{X}$, denoted as Equation 4.

$$\hat{X}[i,j] = \frac{1}{N*d}\sum_{k_1=0}^{N-1}\sum_{k_2=0}^{d-1}\hat{X}_F[k_1,k_2]e^{j2\pi\left(\frac{ik_1}{N}+\frac{jk_2}{d}\right)} \tag{4}$$

Ultimately, LiMTa can obtain the original feature sample X and the frequency feature-missing sample $\hat{X}$.

3) Feature Space Reconstruction: After constructing the original samples and the frequency feature-missing samples, FreqRec pre-trains a powerful feature extraction model by performing the frequency feature reconstruction task.

Feature Extraction Model Design. As illustrated in Figure 3, the feature extraction model is designed as a lightweight architecture for traffic analysis on resource-constrained edge nodes. The first step in the model is a linear transformation that maps the input features into a hidden dimensionality. This transformation allows the model to work in a latent space that is more suitable for subsequent operations. Following the linear transformation, the core of the model consists of a Transformer encoder with 6 layers. Each of these layers includes multihead self-attention with 4 heads and a feed-forward network. This architecture ensures that the model can capture complex dependencies in the traffic data, while maintaining efficiency through residual connections and layer normalization. Dropout is applied to prevent overfitting and enhance generalization, contributing to the model's robustness. After the Transformer encoder, the model employs a convolutional projection layer with a kernel size of 3 to capture local temporal correlations in the sequence data. This convolutional layer helps refine the output representation before it is passed through an optional post-processing step, including layer normalization and dropout, to stabilize the learning process. This compact yet powerful architecture is designed to efficiently handle multitasking traffic analysis while minimizing computational overhead, making it suitable for deployment on edge nodes.

FreqRec Process. To efficiently compute the feature representation of the two samples, FreqRec employs a dual-stream network architecture to reconstruct the features of the frequency feature-missing samples in the feature space, thus training models that can efficiently capture and learn the frequency and temporal features of traffic samples. As illustrated in Figure 2, the fundamental architecture of FreqRec consists of two identical neural networks: the online model, which comprises a feature extraction model f, and the target model, which shares the same structure as the online model but additionally contains a predictor p. The predictor p is composed of a linear layer. The parameters f_m of the target model are the exponential moving average (EMA) [43] of the parameters from the online model.

The encoder f is used to compute the feature representation of the original sample X, and encoder f_m is used to compute the feature representation of the frequency feature-missing sample $\hat{X}$. The predictor p is used to reconstruct the missing frequency information representation. The training objective of this framework is to maximize the similarity between $(X, \hat{X})$ in the feature space, aligning the feature representations of the original sample X and the samples with missing frequency features $\hat{X}$ and achieve the frequency feature reconstruction in feature space. The formula is as follows Equation 5.

$$\mathcal{L}_{ReC} = \min_{p,f}\mathcal{L}_{NCS}(f(X), p\circ f_m(\hat{X})) \tag{5}$$

where $\mathcal{L}_{\text{NCS}}$ is the negative cosine similarity.

$$\mathcal{L}_{NCS}(a,b) = 2 - 2*\frac{<a\cdot b>}{\|a\|_2\|b\|_2} \tag{6}$$

By employing the task of reconstructing frequency features, the pre-trained model can effectively capture both temporal and frequency features of the traffic data. Ultimately, the model enables the learning of high-level semantic features from the raw traffic, thereby providing a richer feature foundation for downstream tasks.

D. Fine-Tuning Stage

To address the limited adaptability of linear fine-tuning and the computational redundancy of existing full fine-tuning and parameter-efficient fine-tuning methods, we propose a

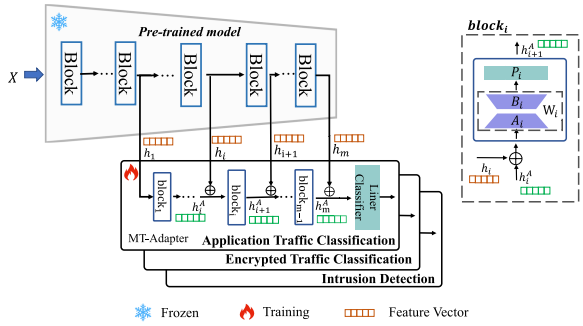


Fig. 4. Architecture of the proposed MT-Adapter for multitask fine-tuning.

lightweight multi-task fine-tuning method called MT-Adapter. Unlike existing fine-tuning methods, MT-Adapter operates independently of the pre-trained model, ensuring that the pre-trained model's forward process is not affected.

Concretely, in the fine-tuning stage, we leverage the hidden features of the pre-trained model to supervise the fine-tuning process of each MT-Adapter module. This supervision enables the MT-Adapter to progressively learn and capture task-specific feature representations layer by layer based on the pre-trained model. Moreover, by freezing the parameters of the pre-trained model and updating only the MT-Adapter's parameters, which account for just 6.37% of the pre-trained model's parameters, the method achieves efficient task-specific fine-tuning while significantly reducing computational costs.

During the inference stage, the MT-Adapter further enhances efficiency by requiring the pre-trained model only once to obtain the hidden layer features. These features are then fed through different MT-Adapter modules to achieve the same performance as full fine-tuning. This approach allows the pre-trained model to be reused across various traffic analysis tasks, reducing the time of model deployment and computation, and significantly lowering the resource costs required.

1) MT-Adapter Design: As illustrated in Figure 4, our proposed MT-Adapter consists of multiple blocks and a linear classification layer, each containing a linear projection layer P and a low-rank matrix projection layer W. Each block takes the corresponding hidden features from the pre-trained model as input, along with the output from the previous block, as shown in Equation 7.

$$h_{i+1}^{A} = P_i(W_i * (h_i^{A} + h_i)) \tag{7}$$

where h_i represents the hidden variable of the i-th layer of the pre-trained model, h_i^A denotes the output of the i-th layer's MT-Adapter, W_i corresponds to the low-rank matrix projection layer of the i-th layer, and P_i is the linear projection layer of the i-th layer. W is expressed as the product of two low-rank matrices B and A. Where $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{r\times d}$, r representing the rank of the matrix. In this paper, the rank r of the matrix W is generally set to the value 8.

Therefore, by aggregating the hidden features of the pre-trained model through multiple layers of blocks, the MT-Adapter can leverage the feature representation capability of the pre-trained model to extract deeper task-specific feature representations. Finally, the results are output through a linear classification layer.

Algorithm 1 MT-Adapter Fine-Tune Process of LiMTa

Input: Data-set $\mathcal{D}$, Classification Layer g, Pre-trained model f;
Initialization: $h_1^A = [0, 0.., 0]$, batchsize ζ = 64.
Init block i, $A_i = N(0, \delta)$, $B_i = 0$, and forward layer P_i
for epoch e = 1,...,E **do**
  for batch b = 1,...,$|\mathcal{D}|/\zeta$ **do**
    Extract a sample (X, y) from $\mathcal{D}_b$
    $f(X) => h_1, h_2 \ldots h_m$
    for i in [1,2,3...,$m-1$] **do**
      $h_{i+1}^A = P_i(B_i A_i * (h_i^A + h_i))$
    end for
    $pred = g(h_m^A + h_m)$
    $loss = CrossEntropy(pred, y)$
  end for
end for

2) Fine-Tuning and Inference Processes: As shown in Algorithm 1, we define a small labeled fine-tuning dataset $\mathcal{D} = \{(X, y)|X \in \mathbb{R}^n, y \in [C]\}$, where X represents the feature data of the traffic after embedding, y is the label of the sample X, and C is the number of categories in the dataset. The input data X first passes through the pre-trained model to compute and store the hidden layer features $H = \{h_1, h_2, \ldots, h_m\}$, where m denotes the number of pre-trained hidden layer layers and h_m corresponds to the output features of the pre-trained model. Then, for each layer of the MT-Adapter, the output is calculated based on Equation 7. Finally, a linear classification layer is added to obtain the output.

During the multi-task inference stage of the model, the input traffic sample X are processed through the pre-trained model to compute and store the hidden layer features. These hidden layer features are then fed into the MT-Adapter modules corresponding to different downstream tasks to obtain the final results. Our method requires only one computation of the pre-trained model to obtain hidden layer features and then feeds these features into different lightweight MT-Adapters to achieve the same model performance as full fine-tuning in solving multiple tasks. By introducing feature aggregation and lightweight modularization, MT-Adapter enables pre-trained models to be effectively reused across various downstream tasks. Simultaneously, by reducing the computational cost of the model, this method allows for the deployment of the model on resource-constrained edge nodes, meeting the demands of real-world applications.

3) Inference Time and Space Complexity Analysis: In this subsection, we analyze the time and space complexity of MT-Adapter across multiple traffic analysis tasks. We define the computational time of the pre-trained model as T_{model}, respectively, with storage spaces denoted as M_{model}. The computation time of a module of the MT-Adapter is $T_{adapter}$, and the storage space is $M_{adapter}$. Notably, a module parameter of MT-Adapter count constitutes only 6.37% of the parameters in

TABLE I
THE STATISTICAL INFORMATION OF THE DATASETS

Traffic Analytics Task Type	Dataset	Labels	Flows	Packets
Application Traffic Classification Task	CrossPlatform-Android	181	27,846	656,044
	CrossPlatform-IOS	125	20,858	707,717
Network Intrusion Detection Task	CICIoT2022	6	476,111	4,536,000
	USTCTFC2016	20	9,853	97,115
Encrypted Traffic Classification Task	ISCXVPN2016	7	2,329	77,163
	ISCXTor2016	8	3,021	80,000

the pre-trained model, rendering it exceptionally lightweight. Therefore, $T_{adapter} \ll T_{model}$ and $M_{adapter} \ll M_{model}$.

To address n edge traffic analytics tasks, conventional fine-tuning inference methods require n times the computation time of the model, as well as n times the storage space of the model, denoted as $n*T_{model}$ and $n*M_{model}$, respectively. In contrast, the model inference time for the MT-Adapter method includes the pre-trained model part and the sum of the inference times of n adapters, with a similar storage space. They are denoted as $T_{model} + n * T_{adapter}$ and $M_{model} + n * M_{adapter}$, respectively. It's worth noting that the MT-Adapter requires a negligible amount of additional storage to temporarily store the hidden features of the pre-trained model, which is insignificant compared to other storage spaces. Overall, when performing multiple tasks (n $>$ 1), the LiMTa method outperforms traditional fine-tuning in terms of both storage and time costs. The lightweight advantage of LiMTa becomes increasingly evident as the number of tasks grows. This analysis is consistent with the experimental results in Section IV-D.

IV. EXPERIMENTS

In this section, we perform extensive experiments on the proposed lightweight multitask traffic analysis framework, LiMTa. First, we introduce the setting of our experiment, including datasets, baseline, evaluation metrics, and so on. Then, to comprehensively measure the effectiveness of LiMTa, we perform the following experiments:

- RQ1: Why and whether LiMTa can train excellent feature extractors and efficiently fine-tune traffic analysis tasks?
- RQ2: Why and whether LiMTa performance improves in various traffic analysis tasks compared to the baseline methods?
- RQ3: Why and whether LiMTa improves the computation time and storage cost?
- RQ4: Why and whether can LiMTa be efficiently fine-tuned with a small set of labeled data?
- RQ5: Why and whether can LiMTa be effectively applied to edge nodes?
- RQ6: Ablation experiments. Why and whether LiMTa components are effective?

A. Experiment Setup

1) Traffic Analysis Task and Dataset: To comprehensively evaluate the effectiveness of LiMTa in addressing multiple edge analytics tasks, we have selected six widely used datasets. These datasets cover a diverse range of task types and encompass a rich variety of application protocol traffic, allowing for a thorough assessment of LiMTa's performance in complex and dynamic edge environments. The tasks and the corresponding datasets are shown in Table I.

Application Traffic Classification Task. These widely used datasets, CrossPlatform-Android [44] and CrossPlatform-IOS [44] datasets, encompass 181 and 125 types of Android and IOS applications, respectively, accurately reflecting the traffic characteristics of users employing diverse applications on mobile devices. These datasets are well-suited for assessing LiMTa's capability to accurately classify multi-application traffic in edge environments.

Network Intrusion Detection Task. The CICIoT2022 [45] dataset simulates a real-world Internet of Things (IoT) environment, encompassing normal traffic and various attack types, with a total of 6 traffic categories. Similarly, the USTCTFC2016 [46] dataset focuses on network intrusion detection, simulating multiple traffic scenarios with 20 traffic categories. Both datasets are instrumental in describing intrusion detection tasks. It provides a platform to assess LiMTa's performance in attack detection and traffic monitoring within edge node environments. This dataset is particularly suited for evaluating the LiMTa framework's ability to handle diverse traffic and identify potential attacks on edge nodes deployed in IoT devices.

Encrypted Traffic Classification Task. The ISCXVPN2016 [47] dataset comprises traffic transmitted through VPNs, with 7 distinct categories. Similarly, the ISCXTor2016 [48] dataset focuses on encrypted Tor network traffic, featuring both normal and attack traffic samples across 8 categories. In edge computing environments, these dataset provides a comprehensive assessment of LiMTa's performance in addressing encrypted traffic analysis tasks.

The six public datasets we use are organized at the session or flow level. In all experiments, we follow common practice in traffic classification and intrusion detection [15], [18], [23], [24] and operate on fixed-length prefixes of each flow. Concretely, for each flow, we only retain the first part of the byte stream and discard the remaining bytes if the flow is longer. This design choice is motivated by two observations: (i) most flows in the considered datasets are short to medium-lived due to their application nature e.g., mobile app requests, VPN sessions used for specific activities, and (ii) prior work has repeatedly shown that the early packets in a connection already contain most of the discriminative information

needed for application identification, intrusion detection, and encrypted traffic classification. In summary, these datasets exhibit a diverse range of task types and closely align with the traffic characteristics of edge nodes, enabling a comprehensive evaluation of LiMTa's effectiveness in handling various traffic analysis tasks within edge environments.

2) Baseline: To comprehensively evaluate the effectiveness of LiMTa, we compare it against SOTA methods in traffic analysis as baselines, including:

- Traditional machine learning approaches, such as AppScanner [16] and FlowPrint [15], which utilize conventional feature extraction and classification techniques;
- Deep learning approaches like DeepPacket [18] and FS-Net [17], which capture complex traffic patterns using deep neural networks;
- Pre-training & fine-tuning approaches such as ET-BERT [23], YaTC [24], and NetMamba [25], which pre-train models on unlabeled tasks before fine-tuning them for specific applications.

Notably, LiMTa-linear represents linear fine-tuning of the pre-trained model; LiMTa-adapter denotes fine-tuning with the MT-Adapter module added to the pre-trained model; LiMTa-all signifies full fine-tuning of the pre-trained model with an additional classification layer.

3) Evaluation Metric: To comprehensively evaluate the model's performance on multi-class traffic analysis tasks, we employed accuracy (ACC) and macro-averaged F1-score (Macro-F1) as evaluation metrics. The macro-averaged F1 score, in particular, considers the precision and recall of each class, making it a more comprehensive performance indicator for datasets, which are calculated as follows:

$$\text{ACC} = \frac{\sum_{i=1}^{N} \mathbf{I}(y_i = \hat{y}_i)}{N} \tag{8}$$

where N is the total number of samples. y_i is the ground truth label for the i-th sample. $\hat{y}_i$ is the predicted label for the i-th sample. $\mathbf{I}(y_i = \hat{y}_i)$ is the indicator function, equal to 1 if $y_i = \hat{y}_i$, and 0 otherwise.

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} \tag{9}$$

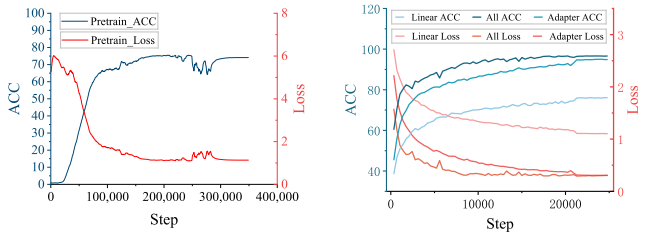
$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c} \tag{10}$$

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{11}$$

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} F1_c \tag{12}$$

where C is the total number of classes. TP_c, FP_c, and FN_c are the true positives, false positives, and false negatives for class c, respectively.

4) Implementation Details: We adopted the widely used self-supervised learning framework, Solo-learn [49]. During the pre-training stage, we employed a Transformer encoder with 6 layers as the pre-training model. We used the CrossPlatform-Android dataset as the unlabeled dataset for pre-training the feature extraction models. We set the batch size to 128 and used a cosine annealing learning rate scheduler



(a) Pre-training process for LiMTa. (b) Fine-tuning process of LiMTa.

Fig. 5. Assessment of the pre-training and fine-tuning validity of LiMTa.

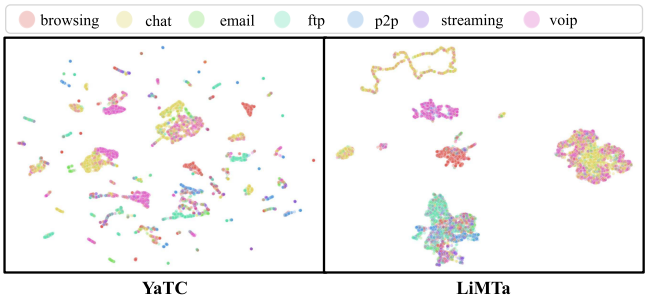


Fig. 6. The visualization of high-dimensional features extracted by the YaTC [24] and LiMTa models.

with an initial learning rate of 0.125 and a weight decay of 1e-5. In the fine-tuning stage, we employed a learning rate of 0.03 and introduced a 5-layer MT-Adapter module. The rank of the BA matrix was set to 8, based on the ablation study presented in Section IV-G, and the warmup ratio was 0.1. All experiments were performed using Pytorch 2.0.0 and on an NVIDIA® 3090 24G GPU. The source code and the pre-trained model checkpoints are made publicly available at https://github.com/liyu779/LiMTa.

B. Assessment of LiMTa Pre-Training and Fine-Tuning Validity

To answer **RQ1**, as shown in Figures 5 (a) and (b), the LiMTa model exhibited good convergence during the pre-training and fine-tuning stages on the CrossPlatform-Android dataset, with a steady increase in accuracy and a gradual decrease in loss. Experimental results demonstrated that linear fine-tuning is less suitable for traffic analysis tasks, while the MT-Adapter module achieved performance comparable to full fine-tuning (Difference of 0.0045 ACC) by updating only a small number of model parameters (6.37% of the pre-trained model). It validated the lack of adaptability of linear fine-tuning and the effectiveness of the MT-Adapter module.

Figure 6 presents the visualization of high-dimensional features extracted by the YaTC and LiMTa models on the ISCXVPN2016 dataset after dimensionality reduction using UMAP [50]. Compared to the YaTC method, the sample points of the LiMTa model were more concentrated, with larger distances observed between different classes. This indicates that the features extracted by the LiMTa model possessed stronger discriminative power. These results confirm that the FreqRec method effectively enhanced the representation of frequency and temporal features in traffic data.

TABLE II
MODEL PERFORMANCE USING TWO EVALUATION PROTOCOLS UNDER DIFFERENT DATASETS. TOP-3 BEST METHODS ARE UNDERLINED. THE BEST METHOD IS BOLDED

Method	CrossPlatform-Android		CrossPlatform-IOS		CICIoT2022		USTCTFC2016		ISCXTor2016		ISCXVPN2016	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1
Machine Learning Based Model												
AppScaner [16]	0.3868	0.3866	0.3205	0.3130	0.7556	0.6938	0.6998	0.6633	0.4034	0.2113	0.7643	0.7256
FlowPrint [15]	0.8739	0.8700	0.8712	0.8603	0.5820	0.4643	0.7992	0.7755	0.1316	0.0306	0.9666	0.9681
Deep Learning Based Model												
Deeppacket [18]	0.8805	0.8138	0.9204	0.9034	-	-	0.9640	0.9641	0.7449	0.7473	0.9329	0.9321
FS-Net [17]	0.4631	0.4628	0.3884	0.3884	0.4391	0.439	0.4542	0.4542	0.7198	0.7198	0.7322	0.7322
Pre-train and Finetune Based Model												
ET-BERT [23]	0.9166	0.9071	0.9105	0.8850	0.9937	0.9937	0.9915	0.9913	0.9980	0.9980	0.9566	0.9565
YaTC [24]	0.9074	0.9074	0.9562	0.9562	0.9584	0.9583	0.9786	0.9785	0.9890	0.9892	0.9772	0.9769
NetMamba [25]	0.9257	0.9260	0.9248	0.9242	0.9944	0.9944	0.9982	0.9982	0.9983	0.9983	0.9826	0.9826
LiMTa-linear	0.7654	0.7654	0.6752	0.6755	0.8743	0.8743	0.8643	0.8649	0.9547	0.9533	0.8829	0.8829
LiMTa-adapter	0.9624	0.9623	0.9522	0.9519	0.9918	0.9918	0.9943	0.9942	0.9966	0.9945	0.9834	0.9835
LiMTa-all	**0.9669**	**0.9669**	**0.9748**	**0.9745**	**0.9949**	**0.9949**	**0.9993**	**0.9992**	**0.9986**	**0.9986**	**0.9870**	**0.9870**

TABLE III
ABLATION ON OTHER SETTINGS IN THE CROSSPLATFORM-ANDROID DATASET

Method	ACC	F1
LiMTa(default)	**0.9624**	**0.9633**
w/o FreqMask	0.1161	0.1138
w/o MT-Adapter	0.7654	0.7654
w/o Position	0.9618	0.9617
w/o Forward P	0.9591	0.9591
w/o Adapter W	0.9240	0.9240

C. Compare LiMTa Performance With Baseline

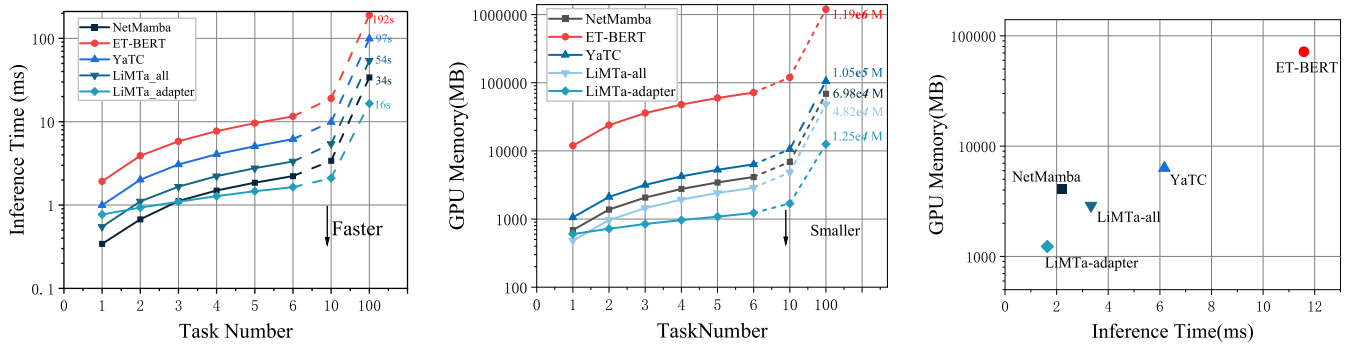
To answer **RQ2**, we conducted comparative experiments against state-of-the-art (SOTA) methods. As shown in Table II, LiMTa consistently outperformed other methods across various datasets and metrics. Although the LiMTA-adapter slightly underperformed compared to LiMTA-all, it achieved remarkable results with only 6.37% of LiMTA-all's trainable parameters. Both LiMTa methods consistently ranked among the top three in all metrics, showcasing SOTA performance.

In terms of the machine learning approach, LiMTa-all and LiMTa-adapter achieved significant improvements of 9.3% (0.8739 $\Rightarrow$ 0.9669) and 8.85 % (0.8739 $\Rightarrow$ 0.9624), respectively, compared to FlowPrint on the CrossPlatform-Android dataset. Unlike FlowPrint and APPScaner methods, which often rely on manual feature engineering, self-supervised learning methods like LiMTa have effectively extracted rich and discriminative features from raw traffic data. Traditional machine learning algorithms depended heavily on feature engineering and could not automate feature extraction, which is the reason for the low performance. In contrast, LiMTa demonstrated the ability to extract frequency features, a capability absent in traditional methods, further highlighting their limitations.

Compared with the deep learning approach, LiMTa-all and LiMTa-adapter significantly outperformed FS-Net and DeepPacket in various evaluation metrics across all datasets. For instance, on the CrossPlatform-Android dataset, LiMTa-all achieved an accuracy improvement of 9.3% (0.8739 $\Rightarrow$ 0.9669) compared to DeepPacket. On the CrossPlatform-IOS dataset, LiMTa models also achieved an accuracy 5.37% improvement (0.9204 $\Rightarrow$ 0.9748). This significant performance improvement was attributed to LiMTa's ability to learn rich traffic features from a massive amount of unlabeled and diverse traffic data, whereas FS-Net and DeepPacket focused on feature extraction for single-task traffic analysis, resulting in relatively limited feature representation capabilities.

In comparison with other pre-trained models, the LiMTa model demonstrated significant advantages across various datasets. Notably, on the CrossPlatform-Android datasets, LiMTa-all achieved accuracy improvements of 5.03% (0.9166 $\Rightarrow$ 0.9669), 7.43% (0.9074 $\Rightarrow$ 0.9669), respectively, compared to ET-BERT and YaTC. This discrepancy could be attributed to the following: ET-BERT learned traffic representations solely from packet payloads; NetMamba and YaTC primarily focused on extracting temporal features and raw space feature reconstruction, neglecting frequency domain information and extracting massive low-level semantic features; In contrast, LiMTa comprehensively incorporated packet headers, temporal and frequency domain features. By reconstructing frequency features within the feature space, LiMTa offered a richer and more accurate representation of network traffic.

In summary, the results presented in Table II conclusively demonstrate that our LiMTa framework achieves state-of-the-art performance across all three traffic analysis tasks, consistently outperforming full-sized pre-trained models while leveraging its lightweight MT-Adapter design to maintain a dramatically lower parameter footprint.



(a) Compare the time cost by different methods in performing different traffic analysis tasks.

(b) Compare the storage cost by different methods in performing different traffic analysis tasks.

(c) Resource cost of different methods in performing 6 tasks.

Fig. 7. Comparing the inference resource cost of different methods.

D. Compare the Inference Resource Cost of LiMTa

To answer **RQ3**, as shown in Figure 7, we evaluated the time and space costs of pre-trained models in six traffic analysis tasks. To better compare the resource overhead of each method, we deployed models that perform different traffic analysis tasks and serially execute them to calculate the storage overhead and time overhead. The inference costs for 10 and 100 traffic analysis tasks were approximated using linear interpolation [51]. The parameter-efficient fine-tuning approach operated similarly to the full fine-tuning approach during deployment and inference, which both require multiple forwards and multiple deployments of the model in multitasking scenarios. Therefore, in this experiment, we used LiMTa-all to measure the inference time and space overhead of the parameter-efficient fine-tuning approach and full fine-tuning.

As shown in Figure 7(a), MT-Adapter demonstrated the shortest inference time as the number of tasks increased. Notably, in the single-task scenario, the computational time of the LiMTa-adapter was slightly higher than that of LiMTa-all due to the additional computational cost introduced by the adapter module. As shown in Figure 7 (b), both LiMTa-adapter and LiMTa-all exhibited lower storage costs. Similarly, in the single-task scenario, the LiMTa-adapter incurred a slightly higher storage cost than the LiMTa-all due to the adapter module and hidden layer feature storage. As shown in Figure 7(c), with six tasks and a batch size of 64, the computational time and storage cost of the LiMTa-adapter were both improved, reducing the time cost by 50.9% and the space cost by 57.4%.

In summary, the LiMTa-adapter effectively reuses the computationally expensive pre-trained model in multi-task scenarios by introducing the lightweight MT-Adapter module, significantly reducing the model's inference time and storage.

E. Evaluate the Label Efficiency of the LiMTa

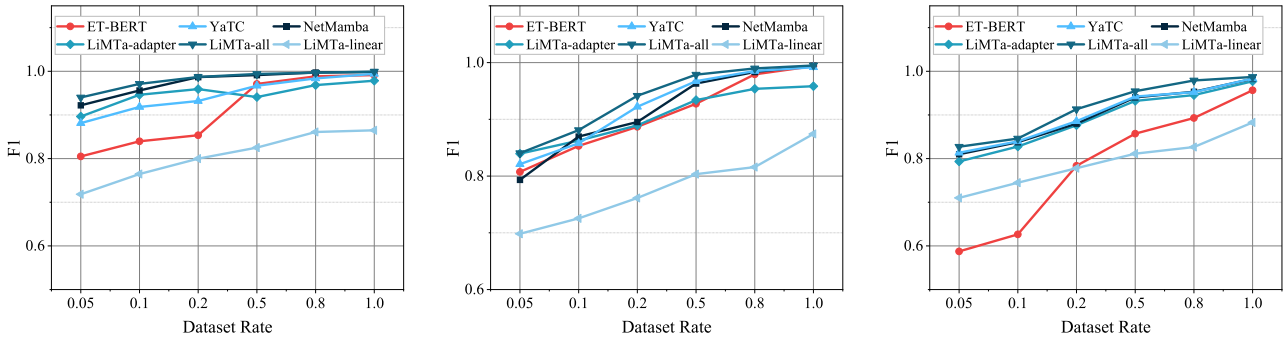
To answer **RQ4**, we evaluated the label efficiency of LiMTa using a subset of the USTCTFC2016 dataset, CICIoT2022 dataset and ISCXVPN2016 dataset. We fine-tuned the pre-trained models of different methods with a small amount of labeled data as a validation of the labeling effectiveness of the methods. As shown in Figure 8, we progressively decreased the dataset size from 100% to only 5%, allowing us to assess LiMTa's performance in scenarios with limited data labels that are very important for edge nodes. Notably, both the LiMTa-adapter and LiMTa-all achieved high F1-score even with small datasets (low data ratios). LiMTa-all achieved the best results on different datasets and different ratios of datasets. In contrast, models like ET-BERT, YaTC, and NetMamba demonstrated inferior performance at low data volumes, particularly ET-BERT, which showed a significantly lower F1-score than the others. These results highlight LiMTa's superior label efficiency, demonstrating its ability to train effectively with minimal labeled data.

In summary, LiMTa maintains its good performance even at lower data sizes. In particular, LiMTa still achieves relatively high performance with limited traffic data volume, which indicates that it has excellent label efficiency. This feature is valuable for real-world edge traffic analysis applications.

F. Evaluate the Edge Node Application of the LiMTa

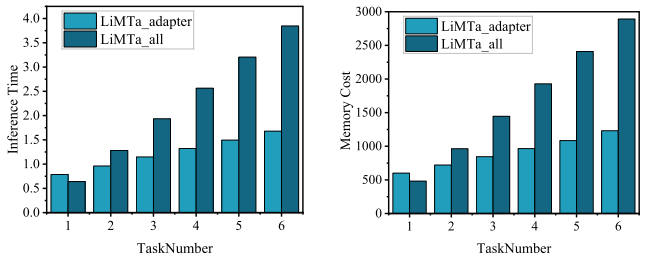
To answer **RQ5**, we compared the computational overhead as well as the storage overhead of full fine-tuning and MT-Adapter on edge devices for different numbers of tasks. Experiments were conducted using the edge node device NVIDIA®JETSON AGX ORIN Developer Kit, equipped with a 2048-core NVIDIA Ampere architecture GPU, 64 Tensor Cores, and 64GB of shared memory. Similar to evaluating model inference overhead, we deployed models that perform different traffic analysis tasks on edge devices and serially execute them to calculate the storage overhead and time overhead.

As shown in Figure 9, experimental results demonstrated that the LiMTa-adapter method offered significant resource efficiency advantage over full fine-tuning (LiMTa-all) on edge nodes. With a task number of 1, the LiMTa-adapter introduced additional adapter parameters, which resulted in slightly higher computational and memory overhead compared to LiMTa-all. However, as the number of tasks increases from 1 to 6, the growth in inference time and memory overhead for LiMTa-adapter was considerably lower than that of LiMTa-all. For example, when the number of tasks was 6, the inference time of LiMTa-adapter was only 1.6785, while LiMTa-all was close to 3.8475; in terms of memory overhead, LiMTa-adapter was 1231M, while LiMTa-all required 2893M.



(a) USTCTFC2016 dataset. (b) CICIoT2022 dataset (c) ISCXVPN2016 dataset

Fig. 8. Compare the labeling efficiency of different methods. For each method, the x-axis shows the ratio of labeled training data used for fine-tuning (from 5% to 100% of the original training set), and the y-axis reports the Macro-F1 score on the full test set.



(a) Inference Time (b) Memory Cost

Fig. 9. Evaluating the time and memory overhead of LiMTa for edge nodes application.

These results highlight the superior scalability and resource efficiency of the LiMTa-adapter, making it better suited for resource-constrained edge node applications, especially in scenarios involving multiple traffic analysis tasks.

G. Ablation Experiment

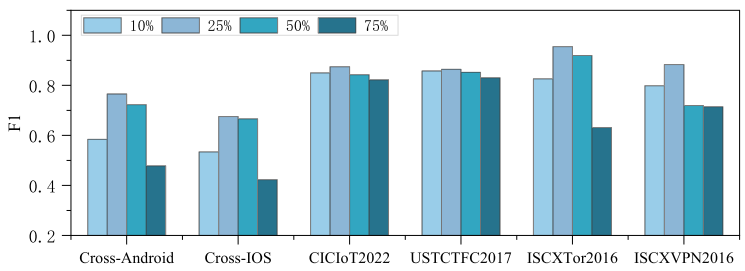
To answer **RQ6**, we performed an ablation study on the proposed LiMTa framework.

Mask Ratio of FreqRec. To evaluate the impact of different masking ratios on the performance of FreqRec, we masked different ratios of frequency domain features and performed a feature reconstruction task to train the feature extraction model. Subsequently, we conducted linear fine-tuning on the pre-trained model while keeping the parameters of the pre-trained model unchanged. The performance of FreqRec was evaluated on six datasets under masking ratios of 10%, 25%, 50%, and 75%. As shown in Figure 10(a), the experimental result indicated that FreqRec performs better under low masking ratios (10% and 25%), demonstrating that the model effectively captured the rich semantic features of traffic data through the frequency domain reconstruction task. However, at a masking ratio of 10%, the model's performance was slightly lower than that at 25%, as the limited amount of masked information provides insufficient supervision signals for the reconstruction task. At higher masking ratios (50% and 75%), the model's performance declined significantly. This was primarily due to the reduced ability to reconstruct frequency domain features when excessive masking was applied. Over-masking led to the loss of critical information, making it challenging for the model to effectively reconstruct features, which ultimately impacted overall performance.

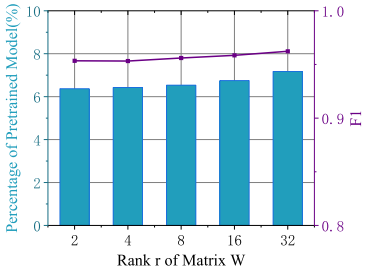
Comparison with the simple FFT. To further validate the effectiveness of FreqRec, we conduct an additional comparison experiment between LiMTa and a simpler FFT baseline. The simple FFT method first applies a Fast Fourier Transform to the raw traffic sequence and then feeds the resulting frequency features into the same backbone architecture for supervised learning. As shown in Figure 10(c), although the FFT baseline can capture basic spectral information, its performance consistently lags behind both LiMTa-all and LiMTa-adapter across all six datasets, for instance, + 47.5% on ISCXTor2016 and + 32.2% on ISCXVPN2016. Although FFT provides a global frequency representation, it lacks the ability to model frequency-component dependencies and local spectral variations. In contrast, the FreqRec learns to recover masked or incomplete frequency components, forcing the model to internalize richer and more robust frequency-domain structures. As a result, LiMTa achieves superior generalization in cross-platform and heterogeneous encrypted traffic scenarios, outperforming the simple FFT baseline.

Rank of the W Matrix. As shown in Figure 10(b), the impact of the rank of the W matrix, a key parameter of the MT-Adapter module, on the fine-tuning performance was analyzed. The experimental results showed that the key parameter rank r in MT-Adapter had a slight impact on model performance and parameter proportion. As rank r increases from 2 to 32, the proportion of pre-trained model parameters rose from approximately 6.37% to nearly 7.17%, while the F1 score steadily improved from around 0.9519 to close to 0.9624, with a small but consistent performance gain. Lower rank values (e.g., $r = 4$ or $r = 8$) achieved excellent performance while maintaining a low parameter proportion, making them well-suited for resource-constrained scenarios. Conversely, higher rank values (e.g., $r = 16$ or $r = 32$) provided further performance enhancements, which were particularly beneficial for scenarios with higher performance requirements and sufficient computational resources. These results demonstrated the flexible trade-off between resource efficiency and performance offered by adjusting the rank r, enabling the MT-Adapter to adapt effectively to diverse application needs.

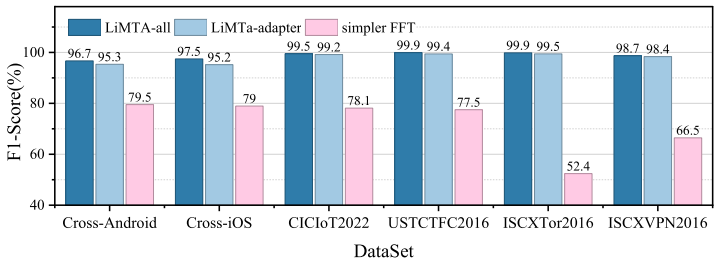
Impact of MT-Adapter Rank on Task Sensitivity. The adapter rank governs the capacity-efficiency trade-off of the



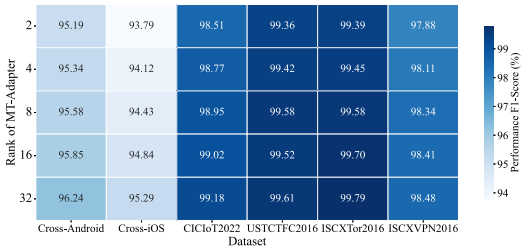
(a) The impact of different masking ratios on the performance of FreqRec in six datasets.



(b) Ablation on different rank r of MT-Adapter in the CrossPlatform-Android dataset.



(c) Comparison of LiMTa with the simple FFT baseline across six benchmark datasets.



(d) Ablation on different rank r of MT-Adapter in six datasets.

Fig. 10. Ablation Experiment on FreqRec and Rank of the W Matrix.Note that the cross-android represents the crossplatform-android dataset, and the Cross-IOS represents the crossPlatform-IOS dataset.

MT-Adapter. To ensure transparency, we conducted a sensitivity analysis on $r \in \{2, 4, 8, 16, 32\}$ across six tasks. As shown in Figure 10, we observed that complex multi-class tasks required a slightly higher rank capacity than simpler tasks to reach optimal performance. We ultimately select $r = 8$ as the final setting, achieving a robust, near-optimal performance while maintaining high parameter efficiency across all heterogeneous traffic analysis tasks.

Other Settings. As shown in Table III, the results of ablation experiments with various settings of LiMTa were analyzed:

- **w/o FreqMask**. In this experiment, the w/o FreqMask variant was designed to perform the original feature space reconstruction task by masking the raw traffic data. The model's accuracy and F1-score dropped significantly to 0.1161 and 0.1138, respectively. This sharp decline occurs because raw bytes in encrypted traffic often resemble high-entropy noise with sparse semantic meaning. Reconstructing masked raw bytes fails to force the model to capture robust dependencies. In contrast, the FreqMask mechanism compels the model to infer these high-level structural features rather than overfitting to local, non-meaningful byte variations, proving its necessity for semantic feature extraction.
- **w/o MT-Adapter**. In this experiment, the MT-Adapter module was removed, and linear fine-tuning was applied to the pre-trained model. The model's performance declined significantly, with both accuracy and F1-score being 0.7654. This result highlights the limitation of linear fine-tuning, which restricts the model to a fixed feature space derived from pre-training. The MT-Adapter is crucial because it introduces a lightweight, learnable transformation that projects these generic pre-trained features into task-specific subspaces. Without the adapter, the model lacks the capacity to align general traffic features with the distinct decision boundaries required for diverse security tasks.
- **w/o Position**. In this experiment, positional embeddings for traffic data were removed. The model's performance decreased slightly, but the change was not significant, with accuracy and F1-score being 0.9618 and 0.9617, respectively. While positional information is generally useful for identifying sequential protocol phases, the performance drop is marginal. This suggests that the frequency features extracted by our FreqRec method are robust and position-agnostic. The model primarily relies on these dominant frequency patterns to distinguish traffic classes, making the explicit positional encoding beneficial but auxiliary rather than critical.
- **w/o Forward P**. The forward module in the MT-Adapter was excluded in this experiment. The model's performance also declined slightly, with both accuracy and F1-score being 0.9591. The linear projection layer P plays a vital role in feature refinement. After the low-rank matrix W projects the features into a task-specific subspace, the layer P is responsible for linearly combining and mapping these features to the optimal output representation for the next block. Removing P restricts the adapter's expressive power, preventing it from fully synthesizing the adapted features, which leads to a suboptimal decision boundary for the classifier.
- **w/o Adapter W**. The low-rank matrix W module from the MT-Adapter was removed. The model's performance declined significantly to 0.9240. This demonstrates the specific contribution of the low-rank matrix W (where $W = BA$) in feature adaptation. While the linear projection P handles dimensionality, W acts as a specific "steering" mechanism that fine-tunes the hidden states.

Removing W limits the adapter's ability to efficiently modulate the pre-trained features, confirming that the low-rank decomposition is key to achieving performance comparable to full fine-tuning with minimal parameters.

V. Discussion

In this section, we discuss some of the limitations of LiMTa in detail so that the reader can further investigate the work. The main aspects included are long-lived and streaming traffic, resource constraints, real-world maintenance, adversarial attacks, interpretability and cross-modal modeling.

A. Applicability to Long-Lived and Streaming Traffic

The current experimental evaluation focuses on fixed-length prefixes of single-session flows, which reflects how existing public datasets are constructed and labeled. In real-world networks, however, long-lived connections and continuous streaming traffic, such as video streaming, long-haul TCP transfers, or persistent IoT telemetry, are also common. These scenarios can be naturally viewed as sequences of fixed-length prefixes of a single evolving flow, which aligns with the data form used in our evaluation. From an architectural perspective, LiMTa remains compatible with such scenarios because both FreqRec and MT-Adapter operate on fixed-length token sequences and do not require complete flows. A long-lived or streaming connection can be split into consecutive windows, e.g., non-overlapping or sliding segments of packets/bytes, each independently embedded, passed through the shared pre-trained encoder, and processed by lightweight task-specific MT-Adapters, enabling low-latency online inference.

B. Hardware Portability and Resource Constraints

Our empirical edge-node evaluation is conducted on a Jetson AGX Orin, but the design of LiMTa does not rely on any platform-specific feature. The dominant computational cost lies in the shared pre-trained encoder, whose complexity scales approximately linearly with sequence length, hidden dimensionality, and model depth, while each MT-Adapter introduces only a small low-rank projection overhead. Because both FreqRec and MT-Adapter operate on fixed-length token sequences, the computational and memory footprint scales predictably with model size and input length. This property makes LiMTa portable across heterogeneous edge platforms, ranging from CPU-only gateways to low-power GPUs and NPUs, where model depth, width, and adapter rank can be flexibly adjusted to match available hardware budgets. To further illustrate this flexibility, we additionally compare FP32 and FP16 numerical precision (reducing memory usage by 62.2%) as well as hidden sizes of 512 vs. 256 (reducing memory usage by 63.4%). Across all these configurations, the accuracy degradation remains within 3% in the CICIoT2022 Dataset, demonstrating that LiMTa enables a controllable accuracy, efficiency trade-off while substantially reducing latency and memory usage under tighter resource constraints.

C. Real-World Model Maintenance and Concept Drift

Beyond resource constraints, a critical challenge for real-world network security systems is concept drift, where the underlying distribution of traffic changes over time, causing performance degradation. The LiMTa framework is inherently designed to address this challenge economically. Since the shared Transformer base model is pre-trained on vast amounts of general traffic data, it captures fundamental, low-level traffic semantics and is expected to exhibit slow drift. Retraining this full base model is computationally prohibitive and only necessary during major shifts in network protocols. Conversely, the lightweight MT-Adapters capture task-specific, high-level representations and are the most sensitive to concept drift. Due to the highly parameter-efficient nature of the Adapters, the system can be maintained by periodically and cheaply retraining only the task-specific Adapters on fresh, labeled data without touching the massive shared encoder weights. This strategy ensures LiMTa remains robust and agile in a dynamic network while minimizing operational costs.

D. Resilience to Adversarial Attacks

One of the key concerns in deploying deep learning models in real-world network traffic analysis tasks is their vulnerability to adversarial attacks. Adversaries may attempt to manipulate the input traffic data in a way that causes the model to misclassify or miss key patterns. While this paper does not specifically focus on adversarial robustness, the lightweight of the LiMTa framework and its use of pre-trained models and feature extraction techniques provide a level of resilience against simple perturbations. However, future work could explore the incorporation of adversarial training methods to enhance the robustness of the model. Techniques like adversarial fine-tuning, robust optimization, and the integration of anomaly detection mechanisms within the model could further fortify LiMTa against sophisticated attacks.

E. Model Interpretability

Another crucial aspect of deploying machine learning models in security-sensitive applications, such as traffic analysis, is model interpretability. Understanding how a model makes decisions is essential for ensuring trust and compliance with security policies. While deep learning models, including those used in LiMTa, are often viewed as "opaque model," our approach using FreqRec enables a degree of interpretability by focusing on frequency-domain features. By reconstructing the frequency features of traffic samples, FreqRec offers a window into how different traffic characteristics influence the model's decision-making. Future work could enhance this interpretability by integrating techniques like attention mechanisms, saliency maps, or layer-wise relevance propagation (LRP) to provide clearer insights into which features are most influential in the model's predictions.

F. Cross-Modal Traffic Models

Current traffic analysis models, including LiMTa, primarily rely on a single modality of data, typically focusing on

packet-level or flow-based features. While these models have shown strong performance, they are limited by the richness of the data they can leverage. In contrast, a cross-modal traffic model would integrate multiple data sources, such as network traffic, system logs and metadata. This multi-source approach could provide a more comprehensive understanding of network activities. By aligning and fusing features from different modalities, a cross-modal model could improve the accuracy and robustness of traffic analysis. The integration of cross-modal data could also enhance the LiMTa framework by providing richer feature representations and enabling more context-aware decision-making, which could further improve both its performance and adaptability in real-world, resource-constrained edge environments.

VI. Conclusion

In this paper, we propose LiMTa, a lightweight multitask traffic analysis framework designed to address the security challenges in distributed networks. More specifically, we propose a novel traffic pre-training method, FreqRec, which achieves high-level semantic feature extraction by reconstructing the frequency features of traffic samples. Moreover, we introduce a pre-trained model fine-tuning method, MT-Adapter, which computes the pre-trained model only once to efficiently perform multiple tasks. By introducing the FreqRec and the MT-Adapter methods, LiMTa efficiently captures both frequency and temporal features while reducing computational and storage costs. The experiments demonstrated SOTA performance on six traffic analysis tasks, and LiMTa significantly exceeded existing SOTA models. Moreover, compared to full fine-tuning, reduced the time cost by 50.9% and the space cost by 57.4%. LiMTa provides a scalable and effective solution for edge network security.

References

[1] J. Waldo, "The jini architecture for network-centric computing," *Commun. ACM*, vol. 42, no. 7, pp. 76–82, Jul. 1999.

[2] J. Ren, D. Zhang, S. He, Y. Zhang, and T. Li, "A survey on end-edge-cloud orchestrated network computing paradigms: Transparent computing, mobile edge computing, fog computing, and cloudlet," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–36, Nov. 2020.

[3] S. Das, A. Silva, and T. S. Eugene Ng, "Rearchitecting datacenter networks: A new paradigm with optical core and optical edge," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 1371–1380.

[4] M. Nazzal, A. Khreishah, J. Lee, S. Angizi, A. Al-Fuqaha, and M. Guizani, "Semi-decentralized inference in heterogeneous graph neural networks for traffic demand forecasting: An edge-computing approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 12, pp. 19400–19416, Dec. 2024.

[5] T. D. Nguyen, P. Rieger, M. Miettinen, and A.-R. Sadeghi, "Poisoning attacks on federated learning-based IoT intrusion detection system," in *Proc. Workshop Decentralized IoT Syst. Secur.*, 2020, Art. no. 23003.

[6] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, and A. Zanella, "IoT: Internet of threats? A survey of practical security vulnerabilities in real IoT devices," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8182–8201, Oct. 2019.

[7] R. Uddin, S. A. P. Kumar, and V. Chamola, "Denial of service attacks in edge computing layers: Taxonomy, vulnerabilities, threats and solutions," *Ad Hoc Netw.*, vol. 152, Jan. 2024, Art. no. 103322.

[8] P. Li, J. Xia, Q. Wang, Y. Zhang, and M. Wu, "Secure architecture for Industrial Edge of Things(IEoT): A hierarchical perspective," *Comput. Netw.*, vol. 251, Sep. 2024, Art. no. 110641.

[9] M. Antonakakis et al., "Understanding the Mirai botnet," in *Proc. 26th USENIX Secur. Symp. (USENIX Secur.)*, 2017, pp. 1093–1110.

[10] E. Ronen, A. Shamir, A. Weingarten, and C. O'Flynn, "IoT Goes nuclear: Creating a ZigBee chain reaction," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 195–212.

[11] S. Herwig, K. Harvey, G. Hughey, R. Roberts, and D. Levin, "Measurement and analysis of Hajime, a peer-to-peer IoT botnet," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2019, Art. no. 23488.

[12] R. Yang et al., "Efficient intrusion detection toward IoT networks using cloud–edge collaboration," *Comput. Netw.*, vol. 228, Jun. 2023, Art. no. 109724.

[13] Z. Diao et al., "EC-GCN: A encrypted traffic classification framework based on multi-scale graph convolution networks," *Comput. Netw.*, vol. 224, Apr. 2023, Art. no. 109614.

[14] F. Li and F. Ye, "Adaptive and lightweight network traffic classification for edge devices," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 4, pp. 2003–2014, Dec. 2022.

[15] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, Art. no. 24412.

[16] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy*, Mar. 2016, pp. 439–454.

[17] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.

[18] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.

[19] A. Alwarafy, K. A. Al-Thelaya, M. Abdallah, J. Schneider, and M. Hamdi, "A survey on security and privacy issues in edge-computing-assisted Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4004–4022, Mar. 2021.

[20] Y. Zhang and Q. Yang, "A survey on multi-task learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 12, pp. 5586–5609, Dec. 2022.

[21] M. G. S. Murshed, C. Murphy, D. Hou, N. Khan, G. Ananthanarayanan, and F. Hussain, "Machine learning at the network edge: A survey," *ACM Comput. Surv.*, vol. 54, no. 8, pp. 1–37, Nov. 2022.

[22] D. Xu et al., "Edge intelligence: Empowering intelligence to the edge of network," *Proc. IEEE*, vol. 109, no. 11, pp. 1778–1837, Nov. 2021.

[23] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[24] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.

[25] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "NetMamba: Efficient network traffic classification via pre-training unidirectional mamba," 2024, *arXiv:2405.11449*.

[26] Z. Lyu, Y. Li, G. Zhu, J. Xu, H. V. Poor, and S. Cui, "Rethinking resource management in edge learning: A joint pre-training and fine-tuning design paradigm," 2024, *arXiv:2404.00836*.

[27] S. Zavrak and M. Iskefiyeli, "Flow-based intrusion detection on software-defined networks: A multivariate time series anomaly detection approach," *Neural Comput. Appl.*, vol. 35, no. 16, pp. 12175–12193, Jun. 2023.

[28] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.

[29] W. Fu, M. Johnston, and M. Zhang, "Low-level feature extraction for edge detection using genetic programming," *IEEE Trans. Cybern.*, vol. 44, no. 8, pp. 1459–1472, Aug. 2014.

[30] K. Yi et al., "Frequency-domain MLPs are more effective learners in time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 76656–76679.

[31] J. Liu and S. Chen, "TimesURL: Self-supervised contrastive learning for universal time series representation learning," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 13918–13926.

[32] J. Fu, J. Fang, J. Sun, S. Zhuang, L. Geng, and Y. Liu, "LoFT: LoRA-based efficient and robust fine-tuning framework for adversarial training," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jun. 2024, pp. 1–8.

[33] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.

[34] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.

[35] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, 2nd Quart., 2019.
[36] D. A. Tedjopurnomo, Z. Bao, B. Zheng, F. M. Choudhury, and A. K. Qin, "A survey on modern deep neural network for traffic prediction: Trends, methods and challenges," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 4, pp. 1544–1561, Apr. 2022.
[37] D. Kwon, H. Kim, J. Kim, S. C. Suh, I. Kim, and K. J. Kim, "A survey of deep learning-based network anomaly detection," *Cluster Comput.*, vol. 22, no. S1, pp. 949–961, Jan. 2019.
[38] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.
[39] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1844–1860.
[40] X.-Y. Chen, L. Han, D.-C. Zhan, and H.-J. Ye, "MIETT: Multi-instance encrypted traffic transformer for encrypted traffic classification," in *Proc. AAAI'25/IAAI'25/EAAI*, 2025, pp. 15922–15929, doi: 10.1609/aaai.v39i15.33748.
[41] B. Mishra and A. Kertesz, "The use of MQTT in M2M and IoT systems: A survey," *IEEE Access*, vol. 8, pp. 201071–201086, 2020.
[42] I. Az, S. Sahin, C. Karakuzu, and M. A. Cavuslu, "Implementation of fft and ifft algorithms in fpga," in *Proc. 3rd Int. Symp. Electr., Electron. Comput. Eng. Symp.*, 2006, pp. 7–10.
[43] Z. Cai, A. Ravichandran, S. Maji, C. Fowlkes, Z. Tu, and S. Soatto, "Exponential moving average normalization for self-supervised and semi-supervised learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 194–203.
[44] J. Ren, D. Dubois, and D. Choffnes, "An international view of privacy risks for mobile apps," Northeastern Univ., Boston, MA, USA, Tech. Rep., 2019.
[45] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Security Trust (PST)*, 2022, pp. 1–11.
[46] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.
[47] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2016, pp. 407–414.
[48] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, Feb. 2017, pp. 253–262.
[49] V. G. T. da Costa, E. Fini, M. Nabi, N. Sebe, and E. Ricci, "solo-learn: A library of self-supervised methods for visual representation learning," *J. Mach. Learn. Res.*, vol. 23, no. 56, pp. 1–6, 2021.
[50] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," 2018, *arXiv:1802.03426*.
[51] T. Blu, P. Thevenaz, and M. Unser, "Linear interpolation revitalized," *IEEE Trans. Image Process.*, vol. 13, no. 5, pp. 710–719, May 2004.

Jiadong Fu received the B.S. degree from Shanghai Maritime University, China, in 2021, and the M.S. degree from the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China, in 2024, where he is currently pursuing the Ph.D. degree. His research interests include model robustness enhancement, edge network security, and model lightweighting.

Jiang Fang received the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, in 2025. He is currently a Post-Doctoral Researcher with the National University of Defense Technology. His research interests include cybersecurity, AI security, and embodied AI security.

Jiyan Sun received the B.S. degree in computer science and technology from Beijing University of Posts and Telecommunications, China, in 2012, and the Ph.D. degree in signal processing from Chinese Academy of Sciences University, Beijing, China, in 2017. She is currently an Assistant Researcher with the Institute of Information Engineering, Chinese Academy of Sciences, China. Her current research interests include the areas of content delivery networks and data center networking.

Shangyuan Zhuang received the B.S. degree from the North China University of Technology, China, in 2021, and the M.S. degree from the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China, in 2024, where she is currently pursuing the Ph.D. degree. Her research interests include adversarial attack and defense and communication network security.

Yinlong Liu received the B.S. degree from Hefei University, China, in 2004, the M.S. degree from the School of Mechanical Electronic & Information Engineering, China University of Mining and Technology, Beijing, China, in 2007, and the Ph.D. degree from the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing, in 2011. His research interests include security in wireless networks and 5G/B5G.

Zhiqiang Lv received the Ph.D. degree in electronic science and technology from Harbin Institute of Technology, Harbin, China, in 2007. He is currently a Professor with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include mobile system security and hardware security.